

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы

Студент гр. 4382	_____	А.Д. Смирнов
Студент гр. 4382	_____	В.А. Росицкий
Студент гр. 4383	_____	Г.А. Гамага
Руководитель	_____	Т.Р. Жангиров

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: А.Д. Смирнов

Группа: 4382

Студент: В.А. Росицкий

Группа: 4382

Студент: Г.А. Гамага

Группа: 4383

Тема практики: Генетические алгоритмы

Задание на практику:

Реализовать генетический алгоритм, решающий следующую задачу: дано N точек в двумерном евклидовом пространстве, необходимо найти координаты левого нижнего угла и длины сторон для M (задается пользователем) квадратов такие, чтобы квадраты суммарно содержали максимальное количество точек внутри и не пересекались между собой. Программа должна иметь графический интерфейс с пошаговой визуализацией, настройкой параметров алгоритма пользователем и графиком изменения функции качества решения от номера популяции, который дополняется с каждым шагом алгоритма.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент гр.4382	_____	А.Д. Смирнов
Студент гр.4382	_____	В.А. Росицкий
Студент гр. 4383	_____	Г.А. Гамага
Руководитель	_____	Т.Р. Жангиров

АННОТАЦИЯ

В данном проекте реализован генетический алгоритм для решения задачи покрытия точек непересекающимися квадратами. Программы написаны на языке *Python* с использованием библиотек *PyQt5* для создания графического интерфейса и *Matplotlib* для отрисовки графиков. Проект разрабатывался с использованием системы контроля версий *git* и платформы *GitHub*.

SUMMARY

This project implements a genetic algorithm to solve the problem of covering points with disjoint squares. The programs are written in Python using the PyQt5 library to create a GUI and the Matplotlib library to draw graphs. The project was developed using the git version control system and the GitHub platform.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	7
2.1. Формирование структуры проекта.....	7
2.2. Разработка прототипа графического интерфейса.....	7
2.3. Реализация ввода исходных точек	8
2.4. Реализация визуализации точек и квадратов	9
2.5. План реализации генетического алгоритма	10
2.6. Реализация генетического алгоритма	11
2.7. Настройка параметров генетического алгоритма.....	13
2.8. Пошаговая визуализация работы алгоритма и график качества.....	13
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	16
4 ОТЗЫВ РУКОВОДИТЕЛЯ	17
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20
ПРИЛОЖЕНИЕ А	21
ПРИЛОЖЕНИЕ Б.....	25

ВВЕДЕНИЕ

Предметной областью учебной практики являются генетические алгоритмы и их применение для решения задач оптимизации. Генетические алгоритмы относятся к эвристическим методам поиска решений и используются в случаях, когда полный перебор вариантов является слишком трудоемким или практически невозможным. В рамках данной работы генетический алгоритм применяется для решения задачи покрытия множества точек на плоскости набором непересекающихся квадратов.

Целью практики является разработка программы с графическим интерфейсом для поиска приближенного решения задачи покрытия точек непересекающимися квадратами. Программа должна позволять задавать входные данные, настраивать параметры генетического алгоритма, визуализировать текущие решения и отображать изменение функции качества по поколениям.

Для достижения цели были поставлены следующие задачи: изучить основные принципы работы генетических алгоритмов, разработать структуру графического интерфейса, реализовать ввод исходных точек, подготовить визуализацию точек и квадратов, создать реализацию генетического алгоритма и операторов отбора, скрещивания, мутации и вычисления функции качества.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы применяются для решения оптимизационных задач, в которых требуется найти достаточно оптимальное решение среди большого числа возможных вариантов. Такие методы используются при планировании, размещении объектов, подборе параметров, маршрутизации и других задачах, где невозможно эффективно проверить все возможные комбинации.

В данной работе рассматривается задача покрытия точек квадратами. На вход подаётся набор точек на плоскости и количество квадратов, которое задаётся пользователем. Требуется подобрать координаты левых нижних углов квадратов и длины их сторон так, чтобы суммарно покрыть максимальное количество точек. При этом квадраты не должны пересекаться между собой. Задача является оптимизационной, так как необходимо максимизировать значение функции качества. В качестве функции качества используется количество покрытых точек с учётом штрафа за пересечение квадратов.

1.2 Задачи практики

В рамках учебной практики необходимо выполнить следующие задачи:

1. Сформировать структуру проекта и выбрать средства разработки.
2. Разработать прототип графического интерфейса.
3. Реализовать ввод исходных данных.
4. Реализовать визуализацию точек и квадратов.
5. Подготовить структуру класса генетического алгоритма.
6. Реализовать генетический алгоритм.
7. Добавить настройку параметров генетического алгоритма через GUI.
8. Реализовать пошаговую визуализацию работы алгоритма и график изменения функции качества.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Формирование структуры проекта

На начальном этапе был создан репозиторий проекта *genetic-optimization-gui*. Для реализации программы был выбран язык *Python 3*. Чтобы не перегружать один файл всей логикой программы, проект был разделён на несколько файлов так, что каждый класс содержится в своём отдельном файле (кроме класса *Square* в силу его небольшого размера). Файл *Constants.py* в отличие от остальных не содержит классов, но содержит константы проекта, связанные с визуализацией, работой алгоритма. В Приложении А представлена общая структура классов разрабатываемой программы (рис. 1).

На UML-диаграмме показана целевая архитектура программы. Класс *MainWindow* отвечает за графический интерфейс, обработку действий пользователя и связь с генетическим алгоритмом. Класс *VisualWidget* используется для визуализации точек и квадратов. Класс *DataUtils* содержит вспомогательные методы для генерации, ручного ввода, загрузки и сохранения данных.

Алгоритмическая часть представлена классом *GeneticAlgorithm*, который управляет процессом поиска. Возможное решение задачи описывается классом *Individual*, который хранит набор объектов *Square*. Класс *Square* хранит в себе координаты левого угла квадрата и длину его стороны. Генетические операторы вынесены в отдельные классы. За отбор родителей отвечает класс *Selection*, за скрещивание – *Crossover*, за мутацию – *Mutation*. Для хранения результатов по поколениям используется список популяций внутри класса *GeneticAlgorithm*.

2.2. Разработка прототипа графического интерфейса

Был разработан прототип графического интерфейса приложения с использованием библиотеки *PyQt5* [1]. Главное окно реализовано в классе *MainWindow*, который наследуется от класса библиотеки *QMainWindow*. Интерфейс был разделён на три основные области. В левой части окна

расположены элементы для ввода исходных данных, настройки параметров алгоритма, таблицы популяций и сохранения популяций. В центральной части находится область визуализации, предназначенная для отображения точек и квадратов, перехода по популяциям и индивидам. В правой части размещён график функции качества, который дополняется по мере работы генетического алгоритма. В Приложении А представлен сам прототип графического интерфейса (рис. 2).

2.3. Реализация ввода исходных точек

Были реализованы три способа задания исходных точек: случайная генерация, загрузка из файла и ручной ввод. В Приложении А представлен блок ввода исходных данных (рис. 3). Их реализации вынесены во вспомогательный класс *DataUtils*. Для хранения точек, введенных пользователем до запуска алгоритма, в классе *MainWindow* используется поле *input_points*, что позволяет сначала подготовить входные данные в интерфейсе, а затем передать в объект класса *GeneticAlgorithm* при запуске алгоритма. При загрузке нового набора точек состояние ранее запущенного алгоритма сбрасывается, так как старая история поколений уже не соответствует новым входным данным. Для предотвращения ошибок были добавлены проверки: программа сообщает пользователю о некорректном формате файла, отсутствии точек и так далее. В Приложении А можно посмотреть пример предупреждения (рис. 4).

Случайная генерация точек запускается с помощью кнопки «Случайные точки». При нажатии кнопки появляется окно *QInputDialog.getInt* для ввода необходимого количества точек для генерации (рис. 5). После генерации точки сохраняются в *input_points*, обновляется информация о количестве точек и вызывается перерисовка области визуализации.

Ручной ввод точек запускается кнопкой «Ввести точки вручную». После нажатия появляется окно *QInputDialog.getMultiLineText* (рис. 6), в котором пользователь может ввести координаты самостоятельно, после чего

программа преобразует введенный текст в список точек и отобразит их в окне визуализации. Это облегчает проверку работы программы на небольших тестовых примерах.

Загрузка точек из файла выполняется кнопкой «Загрузить точки», а сохранение – кнопкой «Сохранить точки». После нажатия на кнопку появляется окно для выбора файла загрузки *QFileDialog.getOpenFileName* или места создания файла сохранения *QFileDialog.getSaveFileName*. Точки могут храниться в файлах JSON или TXT. В JSON-файле точки задаются в виде списка координат, а в TXT-файле каждая строка содержит одну точку.

Также была добавлена возможность сохранения истории популяций с помощью кнопки «Сохранить популяции» в JSON или TXT файл. В файл записываются исходные точки, номера поколений, максимальное и среднее значение функции качества, а также параметры индивидов.

2.4. Реализация визуализации точек и квадратов

Для отображения геометрических объектов был реализован класс *VisualWidget*, который наследуется от *QGraphicsView*. Он отвечает за отрисовку точек и квадратов в центральной части окна, визуализируя графическую сцену. Метод *draw()* очищает текущую сцену и заново отображает точки и квадраты, хранящиеся в полях *squares* и *points*. Точки отображаются в виде небольших окружностей, а квадраты – в виде прямоугольников с одинаковой шириной и высотой. При отрисовке программа проверяет, покрыта ли точка с помощью вспомогательного метода *is_point_covered()*, и выделяет точку другим цветом в случае покрытия. Метод *set_data()* принимает список точек, список квадратов и параметр обновления. Если параметр обновления равен *True*, то метод вызывает *draw()*. Также в области визуализации реализованы масштабирование с помощью колеса мыши, перемещение сцены и отображение координат курсора. Это позволяет легче анализировать расположение объектов на плоскости. В Приложении А представлен пример отображения точек и квадратов (рис. 7).

2.5. План реализации генетического алгоритма

Для решения задачи покрытия точек квадратами был выбран генетический алгоритм, так как задача является оптимизационной и требует подбора большого количества вещественных параметров [2-3]. Входными данными алгоритма являются список точек на плоскости виде пар координат (x, y) , количество квадратов, размер популяции, число поколений, вероятности мутации и скрещивания, метод отбора родителей, а также коэффициенты штрафов и наград, влияющие на функцию приспособленности.

Каждая особь генетического алгоритма представляет собой один вариант расположения квадратов. Если пользователь задает M квадратов, то особь содержит набор $I = [s_1, s_2, \dots, s_M]$, где каждый квадрат задается тройкой параметров $s_i = (x_i, y_i, a_i)$, где x_i и y_i – координаты левого нижнего угла квадрата, а a_i – длина его стороны. Таким образом, хромосома особи представляет собой набор вещественных значений, описывающих координаты и размеры всех квадратов. При инициализации популяции для каждого квадрата координаты генерируются случайным образом вблизи одной из точек с добавлением нормального шума, а ширина определяется как доля от характерного размера поля. Такой подход способствует более быстрой сходимости, так как стартовые решения уже частично покрывают точки.

Качество особи оценивается с помощью функции приспособленности $F = aC^2(I) - bS_{int}(I) - cE(I) - dS(I) - eC_{uncov}(I) - fR(I)$, где $C(I)$ – число покрытых точек, $S_{int}(I)$ – суммарная площадь пересечения квадратов, $E(I)$ – число квадратов, которые ничего не покрывают, $S(I)$ – суммарная площадь квадратов, $C_{uncov}(I)$ – число непокрытых точек, $R(I)$ – сумма расстояний от пустых квадратов до ближайших непокрытых точек, a, b, c, d, e, f – коэффициенты, задаваемые при запуске алгоритма. Такая функция качества позволяет максимизировать покрытие и минимизировать пересечения, а также бороться с образованием бесполезных и слишком крупных квадратов. Чем

больше значение функции приспособленности, тем более удачным считается решение.

Работа генетического алгоритма планируется следующим образом: сначала создаётся начальная популяция случайных особей, затем для каждой особи вычисляется функция приспособленности, после чего выполняются отбор родителей, скрещивание, мутация и формирование нового поколения. В интерфейсе предусмотрен выбор метода отбора родителей. При скрещивании используется двухточечный кроссинговер. При вероятности скрещивания, заданной при запуске алгоритма, хромосомы двух родителей разбиваются на три части в двух случайных точках, и потомки получают комбинацию сегментов от обоих родителей. Если число квадратов меньше трёх или вероятность скрещивания не сработала, потомки являются точными копиями родителей. Мутация осуществляется следующим образом: каждый квадрат в хромосоме с заданной вероятностью подвергается сдвигу или изменению размера. Для пустых квадратов мутация выполняется с большим размахом изменений. Для сохранения решений с максимальным значением функции качества будет использоваться элитизм: определённый процент особей с самым оптимальным решением текущего поколения будет переходить в следующее поколение без изменений. История поколений будет храниться в виде списка поколений внутри класса *GeneticAlgorithm* для того, чтобы реализовать пошаговую визуализацию, построение графика изменения функции качества и возврат к предыдущим поколениям.

2.6. Реализация генетического алгоритма

Для решения задачи покрытия точек квадратами была реализована алгоритмическая часть программы. Основным класс *GeneticAlgorithm* отвечает за хранение входных точек, параметров алгоритма, текущей популяции и истории поколений. При запуске алгоритма вызывается метод *initialize()*, который по исходному набору точек определяет границы рабочей области, создаёт начальную популяцию случайных решений, вычисляет значение

функции приспособленности для каждого индивида и добавляет популяцию в *history*. Каждый индивид представлен объектом класса *Individual* и содержит хромосому – набор объектов *Square*. Один квадрат задаётся тройкой параметров (x, y, a) , где x и y – координаты левого нижнего угла, а a – длина стороны. Таким образом, хромосома индивида представляет собой список квадратов, которые являются возможным вариантом покрытия точек.

Переход к следующему поколению выполняется методом *step()* с помощью стандартных операторов генетического алгоритма, описанных ранее. Из предыдущей популяции выбираются особи с самым оптимальным решением в количестве, определяемом *k_best_percent*, оставшаяся часть популяции используется для выбора родителей. В зависимости от значения поля *selection_method*, устанавливаемого при запуске алгоритме, вызывается соответствующий метод класса *Selection*: *tournament_selection()* — турнирный отбор, *rank_selection()* — ранговый отбор, *roulette_selection()* — рулеточный отбор. Все методы возвращают список пар родителей для скрещивания. Полученные пары передаются в метод *do()* класса *Crossover*, который с вероятностью *crossover_probability* выполняет двухточечное скрещивание, иначе просто копирует родителей. Полученные потомки передаются в метод *do()* класса *Mutation*, который с вероятностью *mutation_probability* для каждого гена выполняет сдвиг или изменение размера квадрата. Новая популяция формируется из элитных особей и мутировавших потомков.

Для новой популяции вычисляются значения функции приспособленности, и она добавляется в историю. Для вычисления значений функции приспособленности используются вспомогательные методы: *calculate_covering* — вычисление числа покрытых точек, *calculate_intersection_area* — вычисление площади пересечения квадратов, *calculate_total_area* — вычисление суммарной площади квадратов, *calculate_far_empty_squares* — вычисление суммы расстояний от пустых квадратов до непокрытых точек.

2.7. Настройка параметров генетического алгоритма

В левой панели главного окна приложения реализован блок элементов управления, позволяющий пользователю гибко задавать все основные параметры генетического алгоритма перед его запуском. Для ввода целочисленных параметров – размера популяции, числа поколений, количества квадратов и так далее – используются компоненты *QSpinBox* из библиотеки *PyQt5*. Для вещественных параметров – вероятности мутации, вероятности скрещивания, штрафа за пересечение квадратов и так далее – применяются *QDoubleSpinBox*. Все параметры ограничены константами, а также имеют определённое начальное значение, которое также задано константами. Константы находятся в файле *Constants.py*. Метод отбора родителей выбирается из выпадающего списка *QComboBox*, содержащего три варианта: «Турнирный отбор», «Рулетка» и «Ранжированный отбор». Все параметры считываются в момент нажатия кнопки «Запустить алгоритм» и передаются в конструктор класса *GeneticAlgorithm*, который инициализирует процесс поиска с учётом заданных настроек. Таким образом, можно проводить серию экспериментов, изменяя параметры и наблюдая за их влиянием на эффективность и скорость сходимости алгоритма.

2.8. Пошаговая визуализация работы алгоритма и график качества

Для наглядного анализа процесса оптимизации в программе реализован механизм пошаговой визуализации. После запуска алгоритма вычисляется только первое поколение популяции, после чего можно последовательно просматривать результаты работы алгоритма на каждом шаге. Внутри класса *GeneticAlgorithm* все сгенерированные популяции сохраняются в списке *history*. При инициализации алгоритма в список добавляется начальная популяция. При каждом вызове метода *step()* вычисляется следующее поколение и добавляется в конец списка.

На центральной панели интерфейса расположены кнопки: «Предыдущий шаг» – программа переходит к прошлому шагу, если сейчас

рассматривается не первое поколение, и перерисовывает визуализацию для предыдущего поколения; «Следующий шаг» – если текущий шаг равен максимальному вычисленному, то выполняется вычисление нового поколения, его добавление в таблицу и смена отображения. Если же текущий шаг меньше максимального вычисленного, то просто сменяется отображение на текущее рассматриваемое поколение; «Перейти к результату» – запускает полный цикл работы алгоритма, вычисляя все поколения до заданного при запуске алгоритма числа. После этого история пополняется, таблица заполняется всеми строками, и отображается последнее поколение. Если поколение было найдено раньше, то программа остановится на поколении с решением.

При переходе на новое поколение виджет *VisualWidget* отображает точки и квадраты, соответствующие выбранному индивиду из этого поколения. При переходе между поколениями выбирается индивидуум с максимальным значением функции приспособленности в популяции. Можно также рассмотреть остальных индивидов с помощью счётчика «Индивид» и вернуться обратно к решению с максимальным значением функции качества в текущей популяции. Кнопка «Посмотреть гены» открывает окно *QMessageBox*, в котором для выбранного индивида выводятся координаты левого нижнего угла и длина стороны для каждого квадрата (рис. 8).

В левой нижней части окна размещена таблица, в которой для каждого вычисленного поколения отображаются его номер, максимальное значение функции приспособляемости и её среднее значение по популяции. При клике на любую строку таблицы происходит переход к соответствующему поколению. В Приложении А можно увидеть, как выглядит таблица (рис. 9)

Для отслеживания динамики работы алгоритма в правой части главного окна реализован график, построенный с помощью библиотеки *Matplotlib*. График имеет панель навигации, которая позволяет пользователю масштабировать, перемещать и сохранять график. На графике отображаются две кривые: максимальное значение функции приспособленности среди всех

особей в поколении (красная линия) и среднее арифметическое функции по всем особям поколения (синяя линия). При инициализации алгоритма график содержит только первое поколение. При каждом переходе на следующий шаг (кнопка «Следующий шаг») или при полном выполнении алгоритма («Перейти к результату») вызывается метод *draw_fitness_plot()*, который полностью перестраивает график на основе всей накопленной истории поколений, хранящейся в *history* класса *GeneticAlgorithm*. В Приложении А прикреплён пример графика функции качества во время работы алгоритма (рис. 10).

Примеры работы программы можно увидеть в Приложении Б.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения учебной практики использовались следующие технологии:

1. *Python 3* – основной язык программирования проекта. Используется для реализации графического интерфейса, вспомогательных функций и генетического алгоритма.

2. *PyQt5* – библиотека для создания графического интерфейса. Используется для построения главного окна, кнопок, полей ввода, таблицы, выпадающих списков и области визуализации.

3. *Matplotlib* – библиотека для построения графиков. Используется для отображения графика изменения функции качества по поколениям.

4. *Git* – система контроля версий. Используется для фиксации изменений, работы по веткам и отслеживания этапов разработки.

5. *GitHub* – платформа для хранения удалённого репозитория проекта и совместной работы над кодом.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв

о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики была разработана программа на языке *Python 3* с графическим интерфейсом, реализующая генетический алгоритм для решения задачи покрытия точек на плоскости непересекающимися квадратами. Все поставленные задачи были успешно выполнены:

- Структура проекта была организована в соответствии с принципами модульности, а использование системы контроля версий *Git* и платформы *GitHub* позволило эффективно вести совместную разработку.
- С помощью библиотеки *PyQt5* был создан графический интерфейс с панелью управления, областью визуализации и графиком качества.
- Реализованы три способа ввода точек: случайная генерация, загрузка из файла (JSON или TXT), ручной ввод.
- Была обеспечена визуализация точек и квадратов конкретного решения с возможностью масштабирования, перемещения и отображением координат.
- Реализован генетический алгоритм с функцией приспособленности, которая учитывает количество покрытых точек, площадь пересечений, пустые квадраты, общую площадь, непокрытые точки и удалённость пустых квадратов от непокрытых областей. Введены три метода отбора, двухточечное скрещивание и мутация сдвигом или изменением размера квадратов индивида, а также механизм элитизма для сохранения наиболее оптимальных решений.
- Пользователь может гибко настраивать генетический алгоритм через графический интерфейс.
- Разработана пошаговая визуализация, позволяющая перемещаться по поколениям, просматривать отдельных индивидов и их гены, и график, построенный с помощью библиотеки *Matplotlib* и

отображающий максимальное и среднее значения функции приспособленности в каждом поколении.

Таким образом, разработанная программа полностью соответствует заданию, цель практики достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Qt for Python [Электронный ресурс]. URL: <https://doc.qt.io/archives/qtforpython-5/> (дата обращения: 04.07.2026).
2. Панченко, Т. В. Генетические алгоритмы [Текст]: учебно-методическое пособие / под ред. Ю. Ю. Тарасевича. — Астрахань: Издательский дом «Астраханский университет», 2007. — 87 [3] с.
3. Вирсански Э., Генетические алгоритмы на Python / пер. с англ. А.А. Слинкиным. — Москва: ДМК Пресс, 2020. — 286 с.: ил. ISBN 978-5-97060-857-9

ПРИЛОЖЕНИЕ А

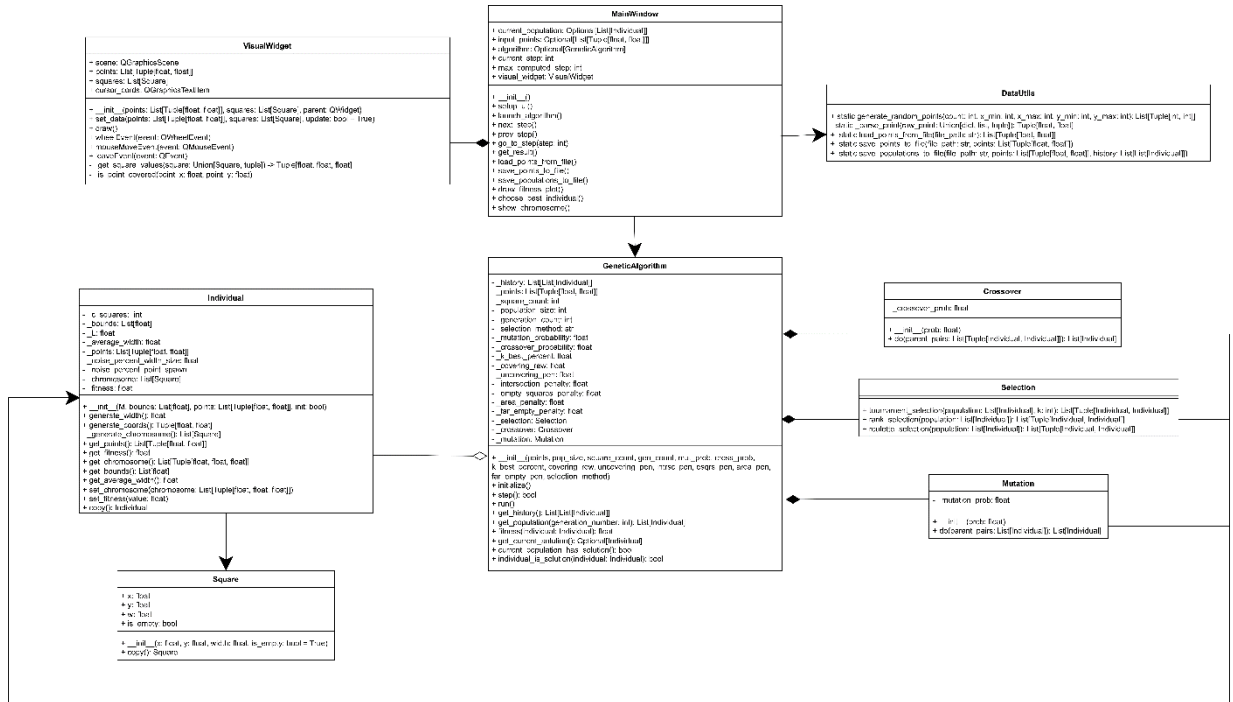


Рисунок 1 – UML-диаграмма классов проекта

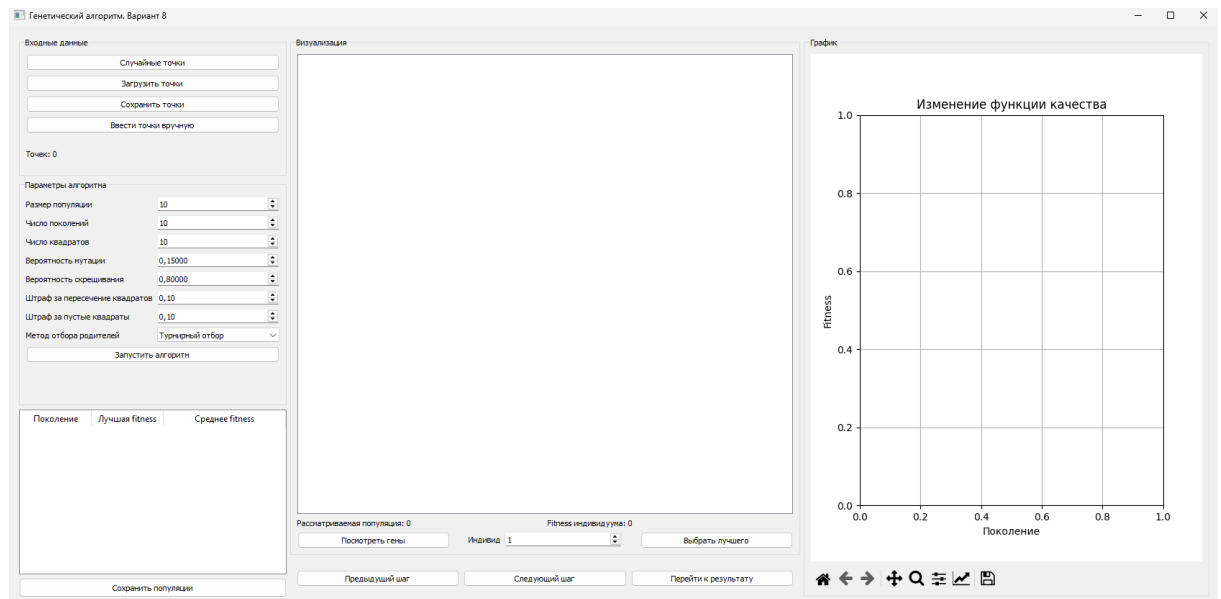
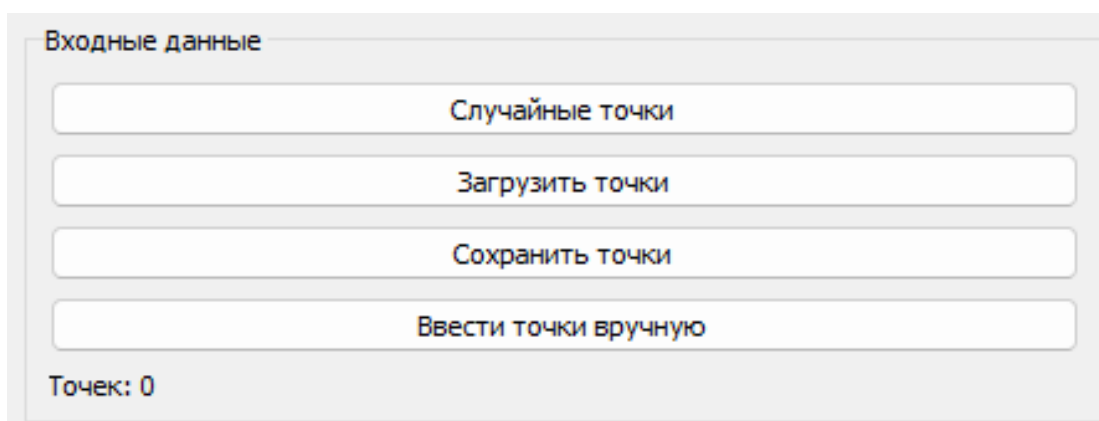


Рисунок 2 – Прототип графического интерфейса



Входные данные

Случайные точки

Загрузить точки

Сохранить точки

Ввести точки вручную

Точек: 0

Рисунок 3 – Блок ввода исходных точек

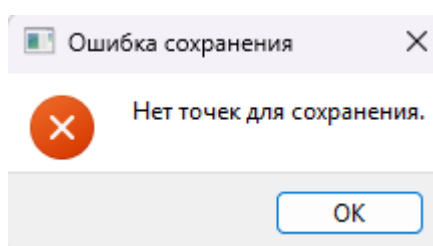
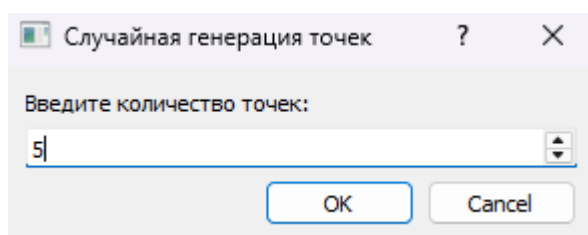


Рисунок 4 – Пример предупреждающего сообщения



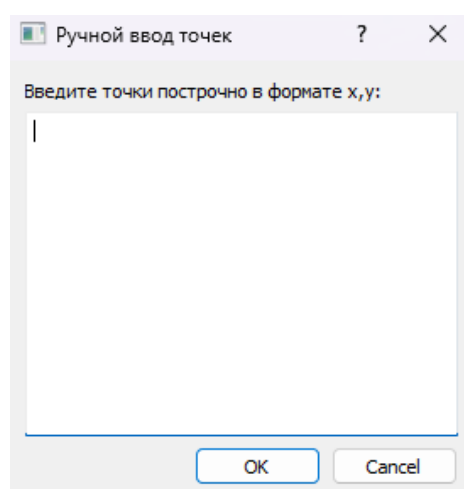
Случайная генерация точек

Введите количество точек:

5

OK Cancel

Рисунок 5 – Окно выбора числа точек для генерации



Ручной ввод точек

Введите точки построчно в формате x,y:

|

OK Cancel

Рисунок 6 – Окно ручного ввода точек

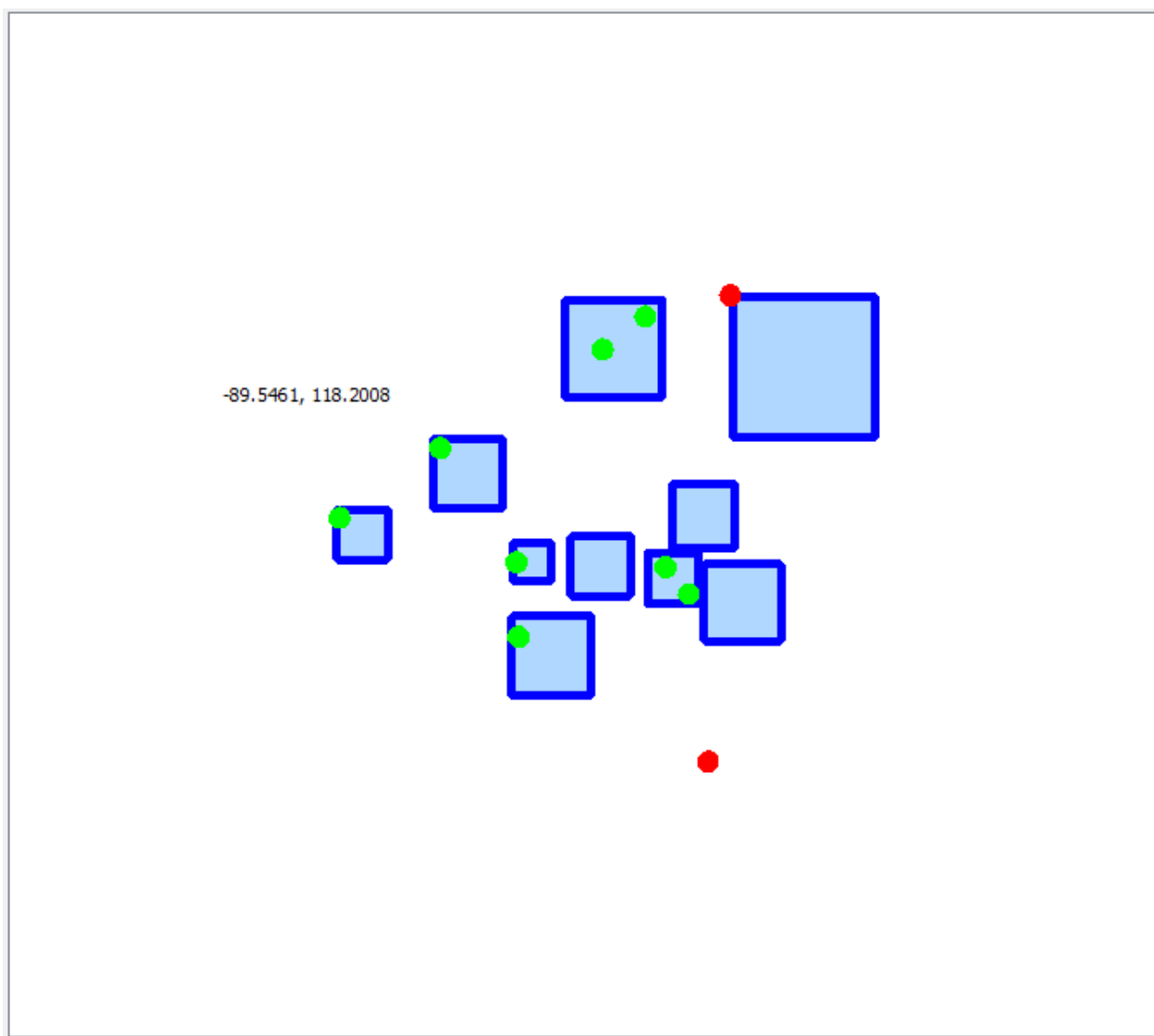


Рисунок 7 – Виджет отображения точек и квадратов решения

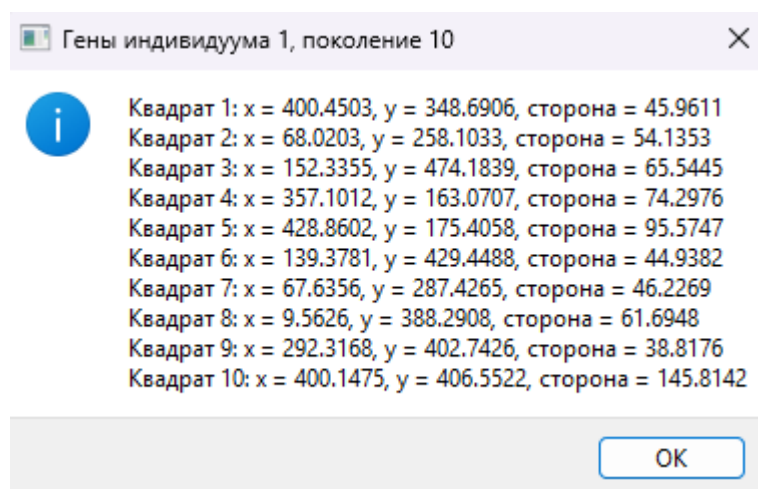


Рисунок 8 – Окно, отображающее гены выбранного индивидуума

Поколение	Лучшая fitness	Среднее fitness
1	-70642.2511	-173690.7391
2	-70642.2511	-157861.3006
3	-26441.0311	-161054.6184
4	-26441.0311	-145698.0207
5	-26441.0311	-126054.6390
6	-26441.0311	-187895.1438
7	-26441.0311	-551692.6178
8	-26441.0311	-443571.6359
9	-26441.0311	-1216086.6551
10	-26441.0311	-438007.1272

Рисунок 9 – Таблица поколений

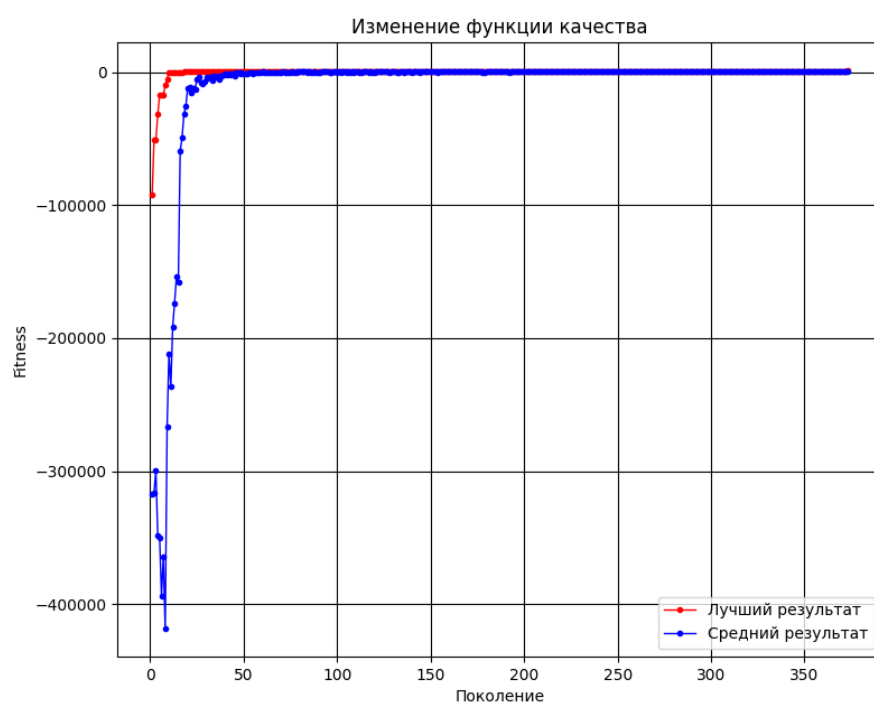


Рисунок 10 – График изменения функции качества

ПРИЛОЖЕНИЕ Б

В этом приложении размещены примеры работы алгоритма. Можно видеть случаи, когда алгоритм нашёл решение за число поколений меньше заданного (рис. 11, рис. 12), и когда не смог найти даже за заданное количество поколений (рис. 13, рис. 14).

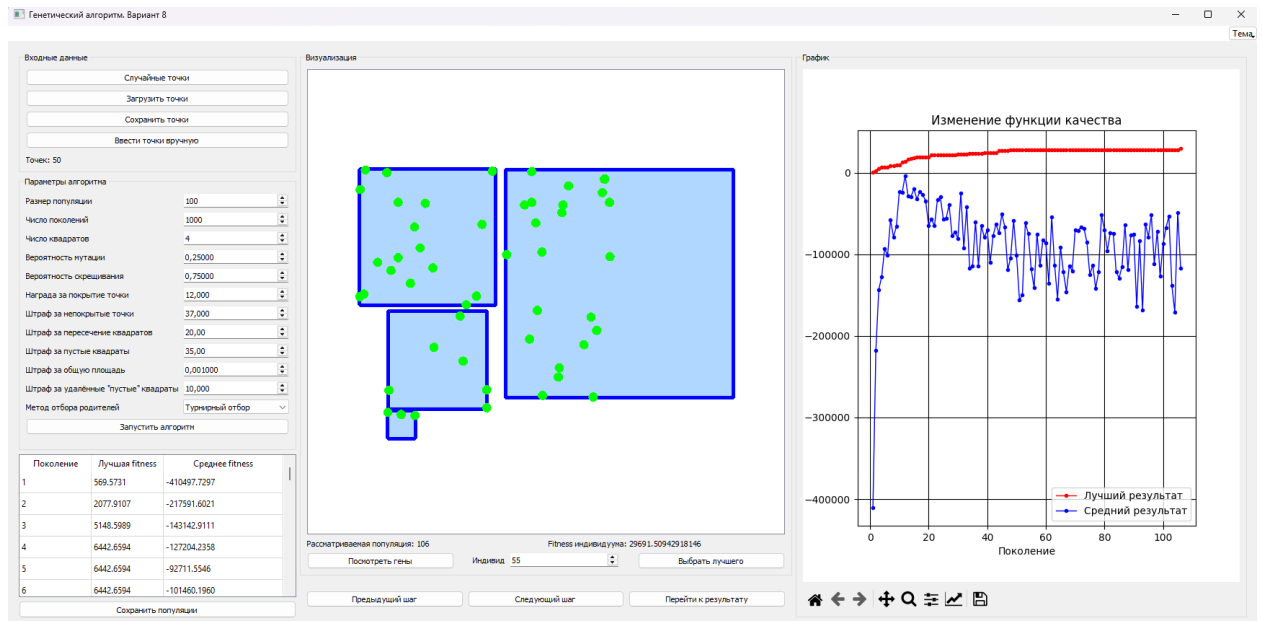


Рисунок 11 – Пример успешного завершения алгоритма с 50 точками

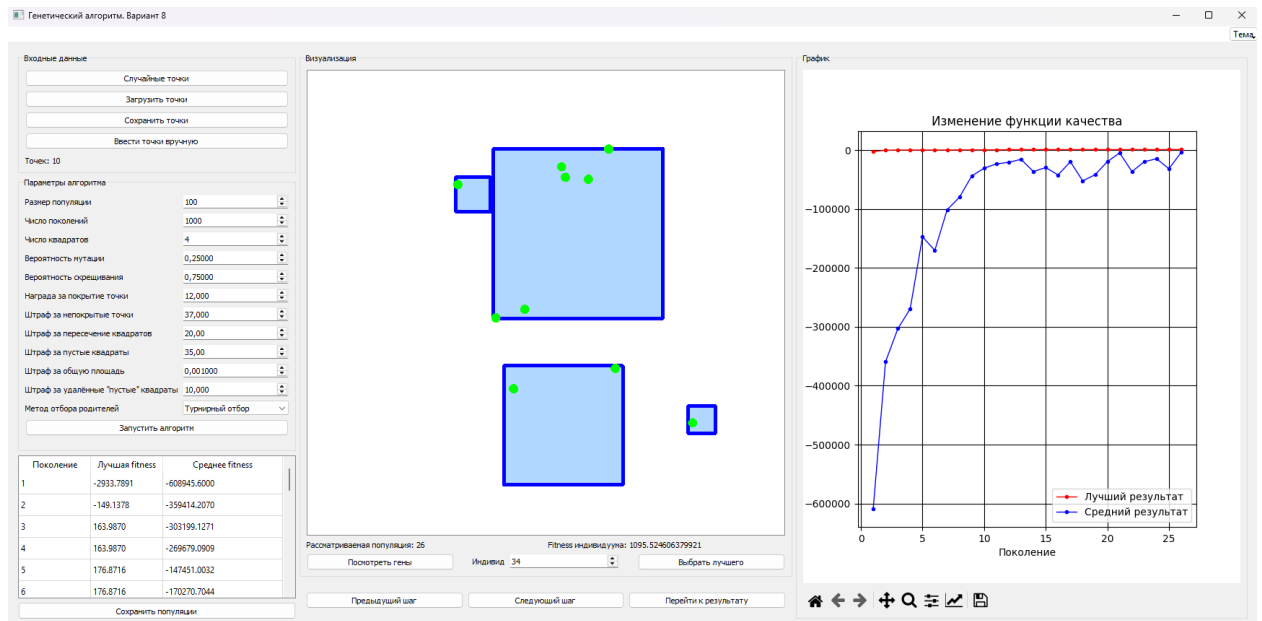


Рисунок 12 – Пример успешного завершения алгоритма с 10 точками

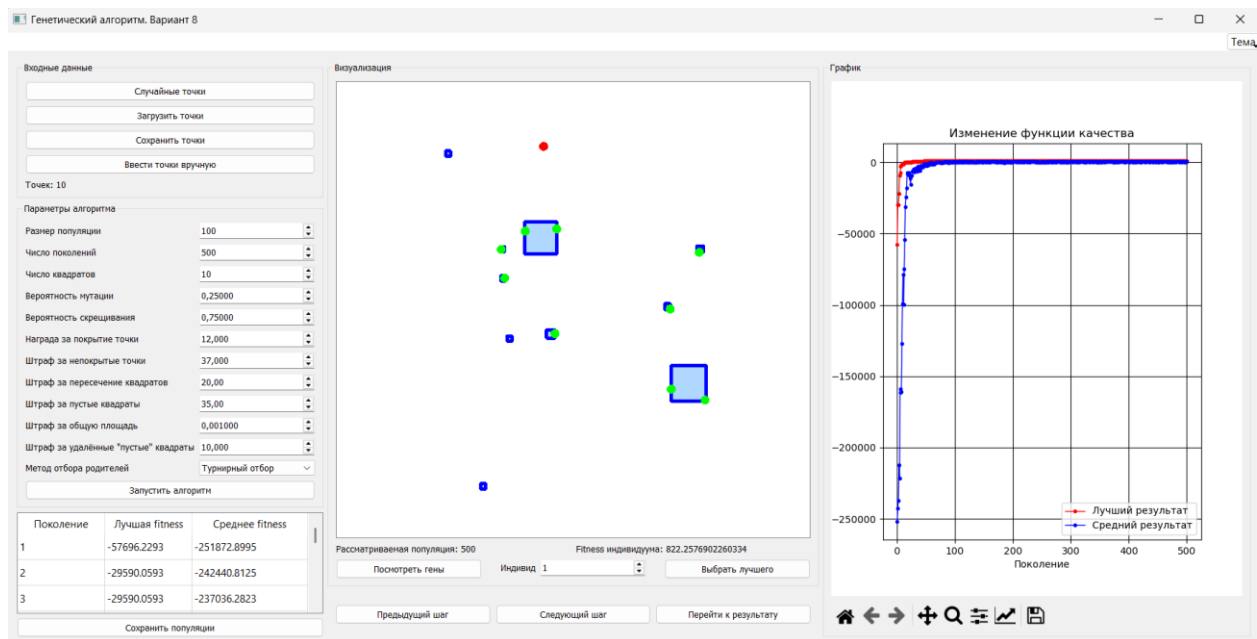


Рисунок 13 – Пример неуспешного завершения алгоритма с 10 точками

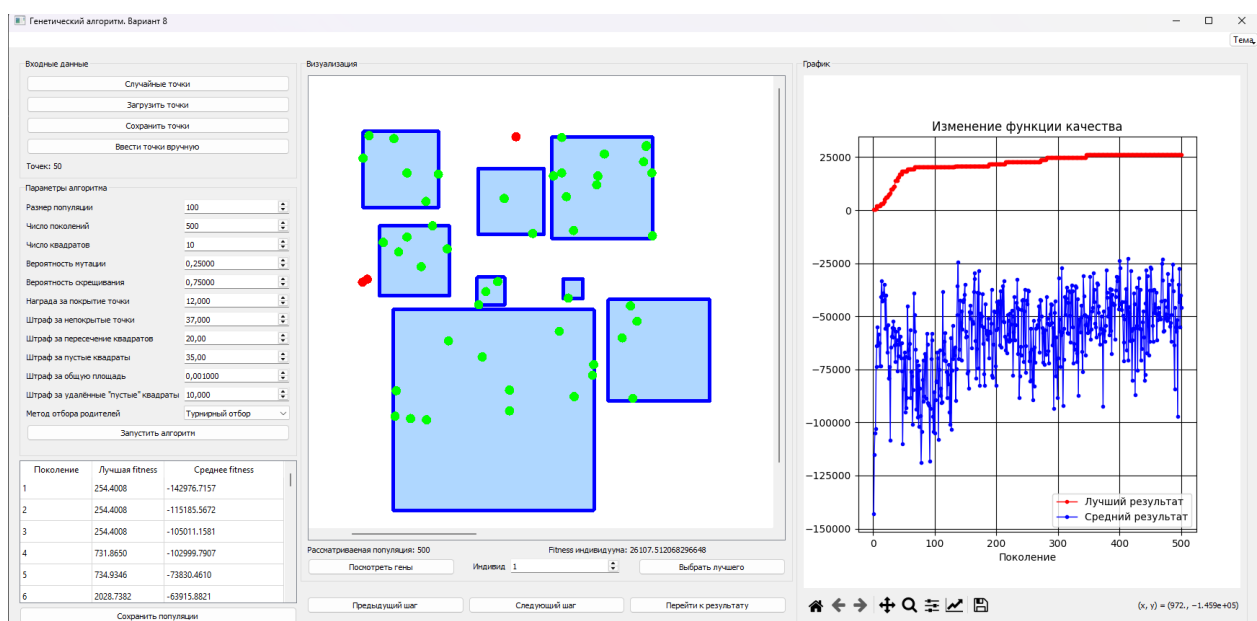


Рисунок 14 – Пример неуспешного завершения алгоритма с 50 точками